

BudgetSync

Smart Personal Finance Management System

Software Requirements Specification (SRS)

Prepared in accordance with IEEE 830 / IEEE 29148 SRS guidelines

Document Version	1.0
Status	Final Draft — For Faculty Evaluation
Prepared For	College / University Software Engineering Project
Category	Personal Finance Management — Web / Cross-Platform Application

Table of Contents

1.	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms & Abbreviations	4
1.4	References	4
1.5	Document Overview	4
2.	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Classes and Characteristics	6
2.4	Operating Environment	6
2.5	Design and Implementation Constraints	7
2.6	Assumptions and Dependencies	7
3.	System Features (Functional Requirements)	8
4.	External Interface Requirements	13
5.	System Architecture Overview	15
6.	Data Requirements	16
7.	Non-Functional Requirements	18
8.	Use Case Summary	20
9.	Appendix	21

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **BudgetSync — Smart Personal Finance Management System**. It is intended to provide a complete, unambiguous, and verifiable reference for the design, development, testing, and evaluation of the system. The document is written for the project development team, the course/faculty evaluators, and any future contributors who extend the system.

This SRS follows the structure recommended by the IEEE 830 / IEEE 29148 standards for software requirements documentation, adapted to a scope appropriate for a college-level software engineering project.

1.2 Scope

BudgetSync is a web-based personal finance management platform that allows an individual user to record and categorize expenses and income, define and track monthly budgets, set and monitor savings goals, maintain a simple overview of personal investments, and view analytical reports and visual insights about their spending behaviour — all from a single, unified dashboard.

The system is scoped for a single-user-per-account model (no shared/family accounts in this phase), and is designed to be realistically implementable by a small student development team within one or two academic semesters, while remaining architecturally sound enough to demonstrate real-world software engineering practice. The system does **not** perform live bank integration, real trading/investment execution, or tax filing in its initial release; these are identified as future-phase extensions (see Section 2.5).

- In scope: user account management, expense & income tracking, budget creation and monitoring, savings goal tracking, manual investment log & portfolio overview, transaction history, analytics/reports dashboard, notifications, and data export.
- Out of scope (current phase): direct bank/API integration, real-time stock trading, multi-currency live conversion, tax computation, and multi-user shared households.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
API	Application Programming Interface
DB	Database
JWT	JSON Web Token — used for authenticated session management
CRUD	Create, Read, Update, Delete — basic data operations
Budget Period	A defined time window (typically monthly) against which spending is tracked relative to a set limit
Transaction	A single recorded financial event: an expense, an income entry, or a transfer

Term	Definition
Dashboard	The main landing screen summarizing a user's financial status at a glance

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.
- Project Brief: *BudgetSync — Smart Personal Finance Management System* (internal project description, provided by the project team).

1.5 Document Overview

Section 2 describes the product at a high level, including its perspective, main functions, target users, and constraints. Section 3 provides detailed functional requirements organized by system feature. Section 4 specifies external interface requirements. Section 5 gives a high-level architecture overview. Section 6 defines the core data requirements. Section 7 specifies non-functional requirements. Section 8 summarizes the primary use cases, and Section 9 contains supporting appendix material.

2. Overall Description

2.1 Product Perspective

BudgetSync is a new, self-contained software product. It is not a modification of an existing system. It is designed as a standard three-tier web application: a browser-based front-end client, a back-end REST API application server, and a relational database for persistent storage. The system is built to be modular so that future phases (e.g., bank-feed integration, mobile app, AI-based spending predictions) can be layered on top of the existing architecture without a full redesign.

2.2 Product Functions

At a high level, BudgetSync provides the following major functions to its users:

- **Account & Profile Management** — secure registration, login, and profile/preferences management.
- **Unified Dashboard** — a single-screen summary of balances, budgets, recent transactions, and alerts.
- **Expense & Income Tracking** — add, edit, categorize, and delete financial transactions.
- **Budget Management** — create category-wise monthly budgets and track utilization in real time.
- **Savings Goals** — define target-based savings goals and track progress toward them.
- **Investment Overview** — manually log investment holdings and view a consolidated portfolio summary.
- **Financial Reports & Analytics** — visual charts and downloadable reports of spending patterns.
- **Notifications & Alerts** — budget-threshold and goal-milestone alerts.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
End User (Primary)	An individual managing their personal finances — the primary and only user role in the current release.	Basic computer/smartphone literacy assumed; no financial or technical expertise required.
Administrator	A system administrator (project maintainer) responsible for monitoring system health, managing categories/master data, and handling support issues.	Moderate — comfortable with admin dashboards and basic system configuration.

2.4 Operating Environment

- **Client side:** Modern web browsers (Chrome, Firefox, Edge, Safari — latest two major versions), responsive design supporting desktop, tablet, and mobile viewports.
- **Server side:** RESTful API service deployable on a Linux-based server or cloud platform (e.g., Node.js/Express or Python/Django/Flask runtime).
- **Database:** A relational database management system (e.g., PostgreSQL or MySQL).

- **Hosting:** Suitable for deployment on standard cloud platforms (e.g., Render, Railway, AWS, Azure) or a local/college server for demonstration purposes.

2.5 Design and Implementation Constraints

- The system must be developed using technologies feasible for a student team within an academic timeline.
- The initial release does not integrate with real banking APIs; all transaction data is entered manually or imported via CSV upload.
- The system must follow secure coding practices for authentication and storage of financial data (e.g., password hashing, encrypted sensitive fields).
- The UI must strictly follow the specified visual theme (Midnight Navy & Emerald Green) for brand and presentation consistency.

2.6 Assumptions and Dependencies

- Users have access to a device with a modern web browser and an internet connection.
- Users are responsible for the accuracy of manually entered financial data.
- The project assumes availability of a hosting environment and database service for deployment/demo purposes.
- Future phases assuming bank-integration will depend on third-party financial data API providers (e.g., Plaid-like services), which are explicitly out of scope for this release.

3. System Features (Functional Requirements)

This section details each major feature of BudgetSync, including its description, priority, and the specific functional requirements (FR) that must be satisfied. Priority levels are: **High** (core / must-have for MVP), **Medium** (important, should be included), and **Low** (nice-to-have, may move to a later phase).

3.1 User Registration & Authentication

Priority:	High
-----------	------

Allows a new user to create a secure account and existing users to log in, ensuring only authenticated users can access personal financial data.

ID	Requirement
FR-01	The system shall allow a new user to register using name, email address, and password.
FR-02	The system shall validate email format and enforce a minimum password strength policy.
FR-03	The system shall hash and securely store user passwords (plain-text storage is prohibited).
FR-04	The system shall allow a registered user to log in using email and password.
FR-05	The system shall issue a session token (e.g., JWT) upon successful login and invalidate it on logout.
FR-06	The system shall provide a 'Forgot Password' flow to reset credentials via a verified email link.
FR-07	The system shall lock or throttle login attempts after multiple consecutive failures.

3.2 Unified Dashboard

Priority:	High
-----------	------

Provides the user a single-screen overview of their overall financial status immediately after login.

ID	Requirement
FR-08	The system shall display total current balance, total monthly income, and total monthly expenses on the dashboard.
FR-09	The system shall display a summary of active budgets with utilization percentage per category.
FR-10	The system shall display the five most recent transactions.
FR-11	The system shall display progress indicators for active savings goals.
FR-12	The system shall display a brief investment portfolio snapshot (total invested value and net change).
FR-13	The system shall visually flag budgets that are near or over their limit using the Emerald/alert color scheme.

3.3 Expense & Income Tracking

Priority: High

Enables users to record, edit, and manage individual financial transactions, forming the core data of the system.

ID	Requirement
FR-14	The system shall allow a user to add a transaction with amount, category, date, type (income/expense), and optional notes.
FR-15	The system shall allow a user to edit or delete an existing transaction.
FR-16	The system shall allow a user to assign a transaction to a predefined or custom category.
FR-17	The system shall allow filtering and searching of transactions by date range, category, and type.
FR-18	The system shall support CSV import of bulk transaction data.
FR-19	The system shall support pagination for large transaction histories.

3.4 Budget Management

Priority: High

Allows users to define spending limits per category and period, and monitors actual spending against those limits.

ID	Requirement
FR-20	The system shall allow a user to create a monthly budget for a specific category with a defined limit amount.
FR-21	The system shall automatically calculate the amount spent against a budget as related transactions are recorded.
FR-22	The system shall display remaining budget and percentage utilized for each active budget.
FR-23	The system shall notify the user when a budget reaches a configurable threshold (e.g., 80%, 100%).
FR-24	The system shall allow a user to edit or delete an existing budget.
FR-25	The system shall allow budgets to roll over automatically at the start of each new period.

3.5 Savings Goals

Priority: Medium

Enables users to define target-based savings objectives and track their progress over time.

ID	Requirement
FR-26	The system shall allow a user to create a savings goal with a target amount, target date, and name.
FR-27	The system shall allow the user to log manual contributions toward a savings goal.

ID	Requirement
FR-28	The system shall calculate and display progress percentage toward each savings goal.
FR-29	The system shall notify the user upon reaching a savings milestone or goal completion.
FR-30	The system shall allow a user to edit or archive a savings goal.

3.6 Investment Portfolio Overview

Priority: Medium

Provides a simplified, manually-maintained overview of a user's investment holdings for consolidated visibility.

ID	Requirement
FR-31	The system shall allow a user to manually add an investment holding with name, type, quantity, and purchase value.
FR-32	The system shall allow a user to update the current value of a holding.
FR-33	The system shall calculate and display total invested value, current value, and net gain/loss.
FR-34	The system shall display a category-wise breakdown of investments (e.g., stocks, mutual funds, fixed deposits).
FR-35	The system shall allow a user to remove a holding from their portfolio.

3.7 Transaction Management & History

Priority: High

Provides a comprehensive, searchable ledger of all recorded financial activity.

ID	Requirement
FR-36	The system shall maintain a chronological, immutable log (audit trail) of all transaction changes.
FR-37	The system shall allow the user to sort transaction history by date, amount, or category.
FR-38	The system shall allow the user to export transaction history as CSV or PDF.

3.8 Financial Reports & Analytics

Priority: High

Presents visual and statistical insights into the user's financial habits to support informed decision-making.

ID	Requirement
FR-39	The system shall generate a category-wise expense breakdown chart (e.g., pie/donut chart) for a selected period.

ID	Requirement
FR-40	The system shall generate an income-vs-expense trend chart over a selectable time range (e.g., last 6 months).
FR-41	The system shall generate a budget-utilization report comparing planned vs. actual spending.
FR-42	The system shall generate a savings growth report showing progress across all active goals.
FR-43	The system shall allow the user to export any report as a PDF document.

3.9 Notifications & Alerts

Priority: Medium

Keeps users informed of important financial events without requiring them to actively check the dashboard.

ID	Requirement
FR-44	The system shall generate an in-app notification when a budget threshold is crossed.
FR-45	The system shall generate an in-app notification for upcoming/overdue recurring transactions, if configured.
FR-46	The system shall generate an in-app notification when a savings goal is completed.
FR-47	The system shall allow the user to mark notifications as read or dismiss them.

3.10 Settings & Profile Management

Priority: Low

Allows users to manage their personal account details and application preferences.

ID	Requirement
FR-48	The system shall allow a user to update their profile information (name, email, password).
FR-49	The system shall allow a user to manage custom transaction categories.
FR-50	The system shall allow a user to set a preferred currency and date format.
FR-51	The system shall allow a user to permanently delete their account and associated data.

4. External Interface Requirements

4.1 User Interfaces

- A responsive web interface accessible via desktop, tablet, and mobile browsers.
- A left/side navigation menu providing access to Dashboard, Transactions, Budgets, Goals, Investments, Reports, and Settings.
- Consistent visual theme across all screens: Midnight Navy (#0F172A) for headers, navigation, and structural elements; Emerald Green (#10B981) for primary actions, accents, positive indicators, and highlights; clean white/off-white backgrounds with dark charcoal body text for readability.
- Data visualizations (charts/graphs) rendered using a standard charting library appropriate to the chosen front-end framework.

4.2 Hardware Interfaces

BudgetSync does not require specialized hardware. It runs on any standard computing device (desktop, laptop, tablet, or smartphone) capable of running a modern web browser with an active internet connection.

4.3 Software Interfaces

Interface	Purpose
Front-end Framework	Renders the UI and communicates with the back-end via REST API calls (e.g., React/Vue/Angular).
Back-end REST API	Exposes endpoints for authentication, transactions, budgets, goals, investments, and reports.
Relational Database	Persists all user, transaction, budget, goal, and investment data (e.g., PostgreSQL/MySQL).
Email Service	Sends account verification, password-reset, and (optionally) alert emails.
Charting Library	Renders analytics charts on the client (e.g., Chart.js, Recharts).

4.4 Communication Interfaces

- All client–server communication shall occur over HTTPS.
- Data shall be exchanged between client and server using JSON over a RESTful API.
- Authenticated requests shall include a bearer token (JWT) in the Authorization header.

5. System Architecture Overview

BudgetSync follows a layered three-tier architecture, separating presentation, application logic, and data persistence. This separation supports maintainability, testability, and future scalability (e.g., adding a mobile client or third-party integrations without altering the core business logic).

Layer	Responsibility	Representative Technology
Presentation Layer	Renders the UI, handles user interaction, calls the API, and displays data/visualizations.	React / Vue + Chart.js, HTML5, CSS3
Application Layer	Implements business logic: authentication, budget calculations, report generation, notification triggers.	Node.js/Express or Python/Django REST Framework
Data Layer	Stores and retrieves persistent data: users, transactions, budgets, goals, investments.	PostgreSQL / MySQL

High-Level Component Flow

A typical request flows as follows: the user interacts with the presentation layer in the browser → the client sends an authenticated HTTPS/JSON request to the application layer's REST API → the application layer validates the request, applies business rules (e.g., recalculating budget utilization), and reads or writes to the data layer → the response is returned to the client and rendered in the UI.

- **Auth Module** — registration, login, token issuance and validation.
- **Transaction Module** — CRUD operations for income/expense entries.
- **Budget Module** — budget CRUD and real-time utilization calculation.
- **Goals Module** — savings goal CRUD and progress tracking.
- **Investment Module** — manual holdings CRUD and portfolio valuation.
- **Analytics Module** — aggregates data for dashboard widgets and reports.
- **Notification Module** — evaluates trigger conditions and creates in-app alerts.

6. Data Requirements

The following core entities represent the primary data model for BudgetSync. This overview is intended to guide future database schema and API design and is not an exhaustive DDL specification.

User

Field	Type	Description
user_id	UUID / PK	Unique identifier for the user
name	String	Full name of the user
email	String, unique	Login email address
password_hash	String	Securely hashed password
currency_pref	String	Preferred display currency
created_at	Timestamp	Account creation date

Transaction

Field	Type	Description
transaction_id	UUID / PK	Unique identifier for the transaction
user_id	UUID / FK	Owning user
category_id	UUID / FK	Associated category
amount	Decimal	Transaction amount
type	Enum	Income or Expense
date	Date	Transaction date
notes	String	Optional description

Budget

Field	Type	Description
budget_id	UUID / PK	Unique identifier for the budget
user_id	UUID / FK	Owning user
category_id	UUID / FK	Budgeted category
limit_amount	Decimal	Maximum allowed spend for the period
period	String	Budget period, e.g. 'Monthly'
start_date	Date	Period start date

SavingsGoal

Field	Type	Description
goal_id	UUID / PK	Unique identifier for the goal

Field	Type	Description
user_id	UUID / FK	Owning user
name	String	Goal name, e.g. 'Emergency Fund'
target_amount	Decimal	Target savings amount
current_amount	Decimal	Amount saved so far
target_date	Date	Target completion date

InvestmentHolding

Field	Type	Description
holding_id	UUID / PK	Unique identifier for the holding
user_id	UUID / FK	Owning user
name	String	Investment name, e.g. 'Index Fund A'
type	String	Stock / Mutual Fund / FD / Other
invested_value	Decimal	Original invested amount
current_value	Decimal	Latest known value

Category

Field	Type	Description
category_id	UUID / PK	Unique identifier for the category
user_id	UUID / FK, nullable	Owner (null for system-default categories)
name	String	Category name, e.g. 'Groceries'
type	Enum	Income or Expense

Entity Relationship Summary

- A **User** has many Transactions, Budgets, Savings Goals, and Investment Holdings (one-to-many).
- A **Category** can be referenced by many Transactions and Budgets (one-to-many).
- Each **Transaction** belongs to exactly one User and one Category.
- Each **Budget** belongs to exactly one User and is scoped to one Category and Period.

7. Non-Functional Requirements

Category	Requirement
Performance	The system shall load the dashboard within 2 seconds under normal network conditions for a user with up to 5,000 transactions.
Security	The system shall encrypt data in transit (HTTPS/TLS) and hash all stored passwords using a strong, salted algorithm (e.g., bcrypt). Sensitive financial fields shall be protected against unauthorized access via role- and token-based access control.
Usability	The UI shall follow a consistent, intuitive layout with the specified color theme, and shall be usable by individuals with no prior financial software experience.
Reliability	The system shall correctly persist all committed transactions and shall not lose recorded data due to normal application-level failures.
Availability	The system shall target at least 99% uptime during the demonstration and evaluation period, excluding scheduled maintenance.
Scalability	The architecture shall support horizontal scaling of the application layer and shall be designed so additional modules (e.g., bank sync) can be added without major rework.
Maintainability	Code shall follow a modular structure with clear separation of concerns (controllers, services, data-access layer) and shall include inline documentation for key modules.
Portability	The web client shall function correctly across major modern browsers and device sizes without requiring browser-specific code.
Compatibility	The system shall support CSV import/export in a widely compatible format (RFC 4180) for interoperability with spreadsheet tools.

8. Use Case Summary

The table below summarizes the primary use cases supported by BudgetSync. Each use case corresponds to one or more functional requirements detailed in Section 3.

Use Case	Actor	Description
Register Account	End User	A new user creates an account to begin using BudgetSync.
Log In	End User	A registered user authenticates to access their dashboard.
Add Transaction	End User	The user records a new income or expense entry.
Create Budget	End User	The user sets a spending limit for a category and period.
Track Budget Utilization	System	The system automatically updates budget usage as new transactions are added.
Create Savings Goal	End User	The user defines a target amount and date for a savings objective.
Log Investment Holding	End User	The user manually records an investment and its value.
View Reports	End User	The user views charts and analytics summarizing their financial activity.
Receive Alert	System	The system notifies the user when a budget or goal threshold is reached.
Export Data	End User	The user exports transactions or reports as CSV/PDF.

9. Appendix

9.1 Default Expense Categories

- Groceries
- Rent / Housing
- Utilities
- Transportation
- Dining & Food Delivery
- Entertainment
- Healthcare
- Education
- Shopping
- Savings & Investments
- Miscellaneous

9.2 Sample Notification Triggers

Trigger Condition	Notification Message (Example)
Budget reaches 80% utilization	"You have used 80% of your Groceries budget for this month."
Budget exceeds 100% utilization	"You have exceeded your Entertainment budget for this month."
Savings goal completed	"Congratulations! You've reached your 'Emergency Fund' goal."
Recurring transaction due	"Reminder: Your recurring Rent payment is due in 2 days."

9.3 Revision History

Version	Date	Description
1.0	Initial Release	First complete draft of the SRS for BudgetSync, covering all core modules for faculty evaluation and student development.

— End of Document —